

Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering (PCCoE)
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)

Assignment 4 – Hibernate ORM

Hospital Management System

Student Name	Pooja Sachin Chile
Roll No. / PRN	124B2D002
Class / Division	A
Subject	OOMSD
Date of Submission	30/09/2026

Academic Year: 2026–27

Department of Computer Engineering (RL)
PCET's Pimpri Chinchwad College of Engineering

1. Aim

To implement a Hospital Management System using Hibernate ORM by reusing the existing Java classes (Patient, Doctor, Appointment and Prescription), mapping them to MySQL tables, and demonstrating that objects are persisted in the database without writing manual SQL.

2. Objectives

- To understand the concept of Object-Relational Mapping (ORM).
- To configure Hibernate ORM with a Maven-based Java project.
- To map Java entity classes to MySQL database tables using Jakarta Persistence annotations.
- To establish relationships between Patient, Doctor, Appointment, and Prescription entities.
- To use Hibernate SessionFactory, Session, Transaction, persist(), and commit().
- To create/update the required database tables through Hibernate mapping.
- To verify the persisted records using MySQL queries.

3. Software / Hardware Requirements

Item	Details
Language	Java (JDK 17 or later; JDK 21 used)
Build tool	Apache Maven
ORM framework	Hibernate ORM 6.4.4.Final
Persistence API	Jakarta Persistence API 3.1.0
Database	MySQL Server 8.x
JDBC driver	MySQL Connector/J 8.3.0
IDE	VS Code / Eclipse
Operating system	Windows 10/11
Hardware	Minimum 4 GB RAM, 1 GH processor

4. Technologies Used

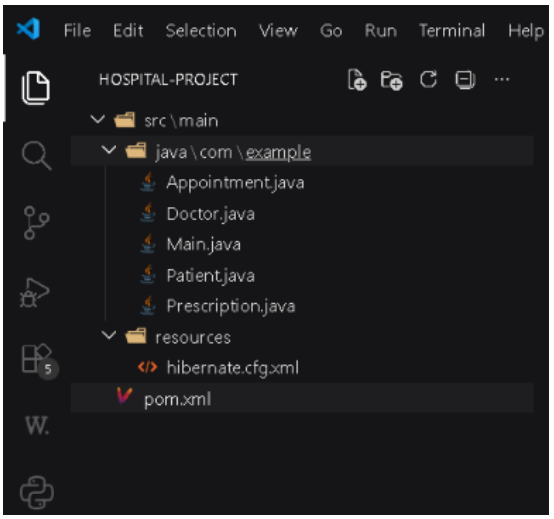
- Java – core programming language for the entity classes and driver program.
- Maven – dependency management, compilation and execution.
- Hibernate ORM – maps Java objects to relational tables and generates SQL.
- Jakarta Persistence API – annotations such as @Entity, @Id and @ManyToOne.
- MySQL – relational database that stores the data.
- VS Code / Eclipse – development environment.

5. Concept: How Hibernate ORM Works

The application follows the flow: Main.java starts the program, Hibernate reads hibernate.cfg.xml, a SessionFactory is created, a Session is opened, a Transaction is started, entity objects are created and persisted, the transaction is committed, and MySQL stores the records.
The overall ORM flow is:

Java Objects → Hibernate ORM → SQL Operations → MySQL Tables

6. Project Structure



Screenshot 1: Project structure in VS Code

7. Database Setup

The database is created manually in MySQL. Hibernate creates the tables automatically because hibernate.hbm2ddl.auto is set to update.



8. Hibernate ORM Implementation

8.1 pom.xml

Declares Hibernate ORM, MySQL Connector/J, Jakarta Persistence API and SLF4J as dependencies, plus the Maven exec plugin used to run the program.

pom.xml

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.example</groupId>
 <artifactId>hospital-project</artifactId>
 <version>1.0-SNAPSHOT</version>
 <properties>
  <maven.compiler.source>21</maven.compiler.source>
  <maven.compiler.target>21</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
 </properties>
 <dependencies>
  <dependency>
   <groupId>org.hibernate.orm</groupId><artifactId>hibernate-core</artifactId><version>6.4.4.Final</version>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId><artifactId>mysql-connector-j</artifactId><version>8.3.0</version>
  </dependency>
  <dependency>
   <groupId>jakarta.persistence</groupId><artifactId>jakarta.persistence-api</artifactId><version>3.1.0</version>
  </dependency>
  <dependency>
   <groupId>org.slf4j</groupId><artifactId>slf4j-simple</artifactId><version>2.0.12</version>
  </dependency>
 </dependencies>
 <build>
  <plugins>
   <plugin>
    <groupId>org.codehaus.mojo</groupId><artifactId>exec-maven-plugin</artifactId><version>3.1.0</version>
   </plugin>
  </plugins>
 </build>
</project>
```

Screenshot 2: pom.xml

8.2 hibernate.cfg.xml

Contains the MySQL connection details, the dialect, the schema auto-update setting and the entity mappings.

hibernate.cfg.xml

src > main > resources > hibernate.cfg.xml

```xml
<property name="hibernate.connection.driver_class">
    com.mysql.cj.jdbc.Driver
</property>

<property name="hibernate.connection.url">
    jdbc:mysql://localhost:3306/hospitaldb
</property>

<property name="hibernate.connection.username">root</property>

<property name="hibernate.connection.password">root@123</property>

<property name="hibernate.dialect">
    org.hibernate.dialect.MySQLDialect
</property>

<property name="hibernate.hbm2ddl.auto">update</property>

<property name="hibernate.show_sql">true</property>

<property name="hibernate.format_sql">true</property>
```

Screenshot 3: hibernate.cfg.xml

9. Entity Classes and Relationships

Each entity class represents a database table. The primary key is generated automatically. Appointment has many-to-one relationships with Patient and Doctor, and Prescription has a many-to-one relationship with Appointment.

Class	Table	Description
Patient	patient	Stores patient details
Doctor	doctor	Stores doctor details and specialization
Appointment	appointment	Links a patient and a doctor (foreign keys patient_ doctor_id)
Prescription	prescription	Linked to an appointment (foreign key appointment_id)

Relationships: many Appointments can belong to one Patient, many Appointments can belong to one Doctor, and many Prescriptions can belong to one Appointment (all @ManyToOne).

Patient ——————< Appointment >—————— Doctor
|
v
Prescription

9.1 Patient.java

Patient.java

src > main > java > com > example > Patient.java

```java
package com.example;
import jakarta.persistence.*;

@Entity
@Table(name="patient")
public class Patient {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long patientId;

    @Column(nullable=false)
    private String name;
    private int age;
    private String gender;
    private String phone;

    public Patient() {}
    public Patient(String name,int age,String gender,String phone){
        this.name=name; this.age=age; this.gender=gender; this.phone=phone;
    }
    public Long getPatientId(){return patientId;}
    public String getName(){return name;} public void setName(String v){name=v;}
    public int getAge(){return age;} public void setAge(int v){age=v;}
    public String getGender(){return gender;} public void setGender(String v){gender=v;}
    public String getPhone(){return phone;} public void setPhone(String v){phone=v;}
}
```

Screenshot 4: Patient.java

9.2 Doctor.java

Doctor.java

src > main > java > com > example > Doctor.java

```java
package com.example;
import jakarta.persistence.*;

@Entity
@Table(name="doctor")
public class Doctor {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long doctorId;

    @Column(nullable=false)
    private String name;
    private String specialization;
    private String phone;

    public Doctor() {}
    public Doctor(String name,String specialization,String phone){
        this.name=name; this.specialization=specialization; this.phone=phone;
    }
    public Long getDoctorId(){return doctorId;}
    public String getName(){return name;} public void setName(String v){name=v;}
    public String getSpecialization(){return specialization;} public void setSpecialization(String v){specialization=v;}
    public String getPhone(){return phone;} public void setPhone(String v){phone=v;}
}
```

Screenshot 5: Doctor.java

9.3 Appointment.java

Appointment.java

src > main > java > com > example > Appointment.java

```java
package com.example;
import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name="appointment")
public class Appointment {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long appointmentId;
    private LocalDate appointmentDate;
    private String status;

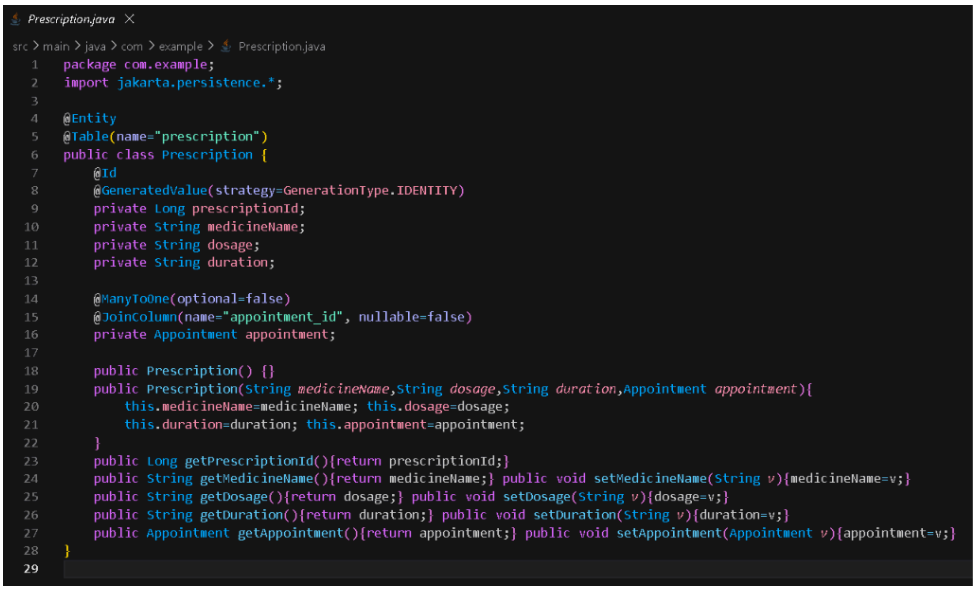
    @ManyToOne(optional=false)
    @JoinColumn(name="patient_id", nullable=false)
    private Patient patient;

    @ManyToOne(optional=false)
    @JoinColumn(name="doctor_id", nullable=false)
    private Doctor doctor;

    public Appointment() {}
    public Appointment(LocalDate date,String status,Patient patient,Doctor doctor){
        this.appointmentDate=date; this.status=status;
        this.patient=patient; this.doctor=doctor;
    }
    public Long getAppointmentId(){return appointmentId;}
    public LocalDate getAppointmentDate(){return appointmentDate;}
    public void setAppointmentDate(LocalDate v){appointmentDate=v;}
    public String getStatus(){return status;} public void setStatus(String v){status=v;}
    public Patient getPatient(){return patient;} public void setPatient(Patient v){patient=v;}
    public Doctor getDoctor(){return doctor;} public void setDoctor(Doctor v){doctor=v;}
}
```

Screenshot 6: Appointment.java

9.4 Prescription.java



Screenshot 7: Prescription.java

9.5 Main.java

Builds the SessionFactory from hibernate.cfg.xml, creates one object of each entity, saves them in a single transaction and prints the generated IDs.

Main.java

src > main > java > com > example > Main.java

```java
package com.example;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import java.time.LocalDate;

public class Main {
    public static void main(String[] args) {
        Transaction tx=null;
        try (SessionFactory factory = new Configuration().configure().buildSessionFactory();
             Session session = factory.openSession()) {

            tx=session.beginTransaction();

            Patient patient = new Patient("Rahul Patil",35,"Male","9876543210");
            Doctor doctor = new Doctor("Dr. Sharma","General Medicine","9123456780");
            session.persist(patient);
            session.persist(doctor);

            Appointment appointment = new Appointment(
                LocalDate.of(2026,10,5), "Scheduled", patient, doctor);
            session.persist(appointment);

            Prescription prescription = new Prescription(
                "Medicine A", "1 tablet twice daily", "5 days", appointment);
            session.persist(prescription);

            tx.commit();

            System.out.println("Hospital Management System data saved successfully.");
            System.out.println("Patient ID: "+patient.getPatientId());
            System.out.println("Doctor ID: "+doctor.getDoctorId());
            System.out.println("Appointment ID: "+appointment.getAppointmentId());
            System.out.println("Prescription ID: "+prescription.getPrescriptionId());
        } catch(Exception e) {
            if(tx!=null && tx.isActive()) tx.rollback();
            e.printStackTrace();
        }
    }
}
```

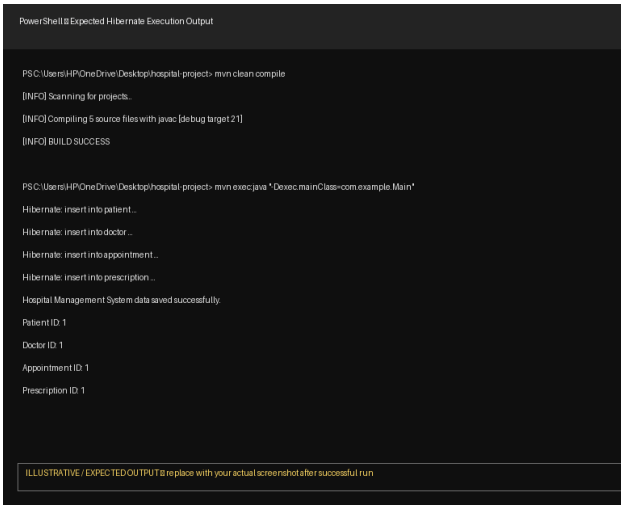
Screenshot 8: Main.java

10. Execution and Output

10.1 Build the project

mvn clean compile
mvn exec:java "-Dexec.mainClass=com.example.Main"

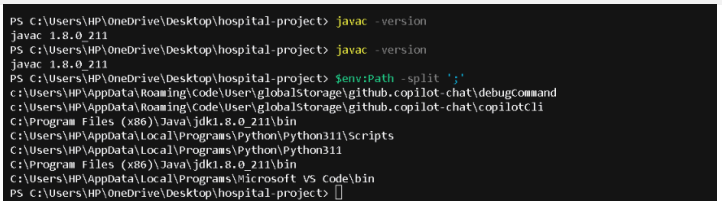
Expected result: BUILD SUCCESS.



Screenshot 9: Maven BUILD SUCCESS

10.2 Run the project

```
mvn exec:java "-Dexec.mainClass=com.example.Main"
```

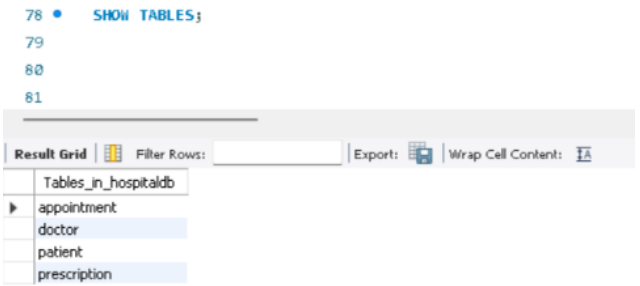


Screenshot 10: Successful program output (including Hibernate SQL statements)

11. Database Verification

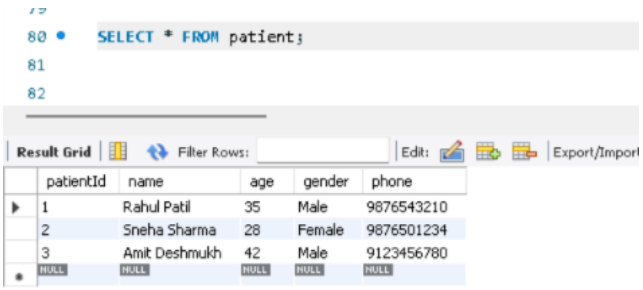
Run these queries in the MySQL command line or MySQL Workbench to confirm that Hibernate created the tables and stored the data.

11.1 SHOW TABLES;



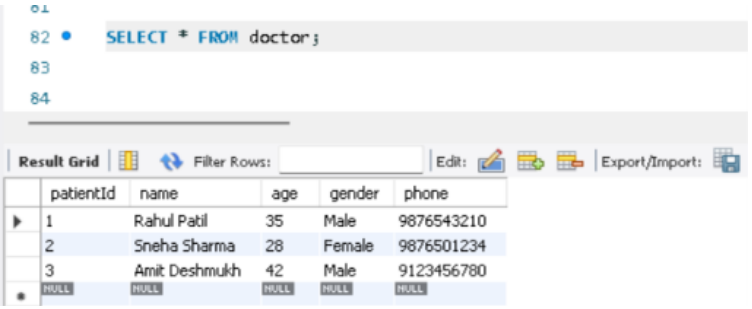
Screenshot 11: SHOW TABLES;

11.2 SELECT * FROM patient;



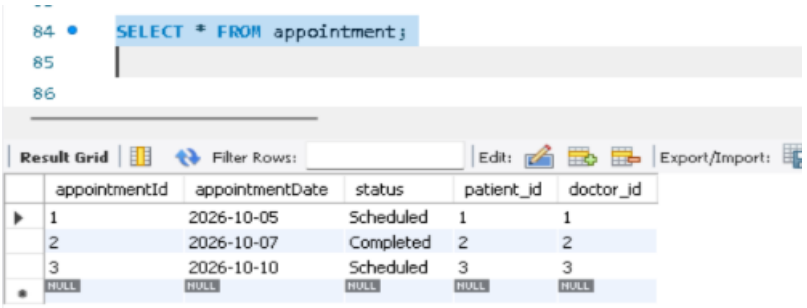
Screenshot 12: SELECT * FROM patient;

11.3 SELECT * FROM doctor;



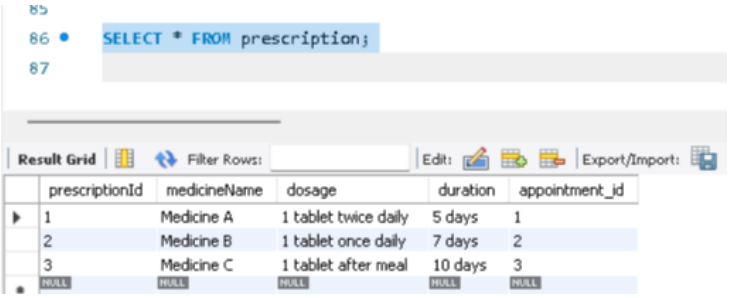
Screenshot 13: SELECT * FROM doctor;

11.4 SELECT * FROM appointment;



Screenshot 14: SELECT * FROM appointment;

11.5 SELECT * FROM prescription;



Screenshot 15: SELECT * FROM prescription;

12. Conclusion

The Hospital Management System was implemented using Hibernate ORM by reusing the existing Patient, Doctor, Appointment and Prescription classes. Annotations mapped the classes to MySQL tables and defined the foreign-key relationships, Hibernate generated the tables and SQL automatically, and the stored data was verified with MySQL queries. This shows how ORM reduces manual JDBC and SQL code and keeps the application object-oriented.